

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Облачные и туманные вычисления

Лабораторная работа № 3

«Шлюз «Умной фабрики» (Симуляция
Edge/Fog)»

Выполнили

Баянов Равиль Динарович

Группа № Р3434

Преподаватель: Перл Ольга Вячеславовна

г. Санкт-Петербург 2025

Содержание

Содержание	2
Ссылка на репозиторий проекта	3
Настройка инфраструктуры	Ошибка! Закладка не определена.
Настройка шлюза	Ошибка! Закладка не определена.
Запуск симуляции.....	7
Наблюдение.....	Ошибка! Закладка не определена.
Вывод.....	10

Ссылка на репозиторий проекта

<https://github.com/RavvChek/ITMO/tree/main/4%20year/%D0%9E%D0%B1%D0%BB%D0%B0%D1%87%D0%BD%D1%8B%D0%B5%20%D0%B8%20%D1%82%D1%83%D0%BC%D0%B0%D0%BD%D0%BD%D1%8B%D0%B5%20%D0%B2%D1%8B%D1%87%D0%B8%D1%81%D0%BB%D0%B5%D0%BD%D0%B8%D1%8F/lab3>

Исходный код

Код

```
import json
import os
import random
import time
from datetime import datetime, timezone

import paho.mqtt.client as mqtt

def env_int(name: str, default: int) -> int:
    try:
        return int(os.getenv(name, str(default)))
    except ValueError:
        return default

def env_float(name: str, default: float) -> float:
    try:
        return float(os.getenv(name, str(default)))
    except ValueError:
        return default

MQTT_HOST = os.getenv("MQTT_HOST", "localhost")
MQTT_PORT = env_int("MQTT_PORT", 1883)
MQTT_TOPIC = os.getenv("MQTT_TOPIC", "factory/temperature")
DEVICE_ID = os.getenv("DEVICE_ID", "sensor-1")

INTERVAL_SECONDS = env_float("INTERVAL_SECONDS", 1.0)
TEMP_MIN = env_float("TEMP_MIN", 55.0)
TEMP_MAX = env_float("TEMP_MAX", 95.0)

def make_temperature() -> float:
    if random.random() < 0.15:
        return random.uniform(81.0, TEMP_MAX)
    return random.uniform(TEMP_MIN, 79.5)

def main() -> None:
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.connect(MQTT_HOST, MQTT_PORT, keepalive=60)
    client.loop_start()

    print(
        f"[edge-sensor] Publishing to mqtt://{MQTT_HOST}:{MQTT_PORT} topic={MQTT_TOPIC} "
        f"device_id={DEVICE_ID} interval={INTERVAL_SECONDS}s"
    )

    try:
        while True:
            temperature = round(make_temperature(), 2)
            payload = {
                "device_id": DEVICE_ID,
                "temperature": temperature,
                "ts": datetime.now(timezone.utc).isoformat(),
            }
            msg = json.dumps(payload, ensure_ascii=False)
            result = client.publish(MQTT_TOPIC, msg, qos=0, retain=False)
            status = "ok" if result.rc == mqtt.MQTT_ERR_SUCCESS else f"rc={result.rc}"
            print(f"[edge-sensor] {status} {msg}")
            time.sleep(INTERVAL_SECONDS)
    except KeyboardInterrupt:
        pass
    finally:
        client.loop_stop()
        client.disconnect()

if __name__ == "__main__":
    main()
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV PYTHONUNBUFFERED=1
CMD ["python", "app.py"]
```

Mosquitto

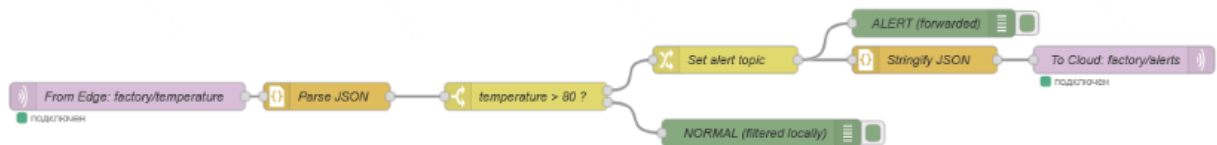
edge

```
listener 1883
allow
-
anonymous true
persistence false
log_
type information
```

cloud

```
listener 1883
allow
-
anonymous true
persistence false
log_
type information
```

Node-Red



Запуск приложения

Docker-compose.yml

```
services:
  edge-mqtt:
    image: eclipse-mosquitto:2
    container_name: lab3-edge-mqtt
    volumes:
      - ./mosquitto-edge/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    ports:
      - "1883:1883"
    networks:
      - factory-net

  cloud-mqtt:
    image: eclipse-mosquitto:2
    container_name: lab3-cloud-mqtt
    volumes:
      - ./mosquitto-cloud/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    ports:
      - "1884:1883"
    networks:
      - factory-net

  nodered:
    image: nodered/node-red:latest
    container_name: lab3-nodered
    environment:
      - FLOWS=flows.json
    ports:
      - "1880:1880"
    volumes:
      - ./nodered:/data
    depends_on:
      - edge-mqtt
      - cloud-mqtt
    networks:
      - factory-net

  edge-sensor:
    build:
      context: ./edge-sensor
    container_name: lab3-edge-sensor
    environment:
      - MQTT_HOST=edge-mqtt
      - MQTT_PORT=1883
      - MQTT_TOPIC=factory/temperature
      - DEVICE_ID=sensor-1
      - INTERVAL_SECONDS=1
      - TEMP_MIN=55
      - TEMP_MAX=95
    depends_on:
      - edge-mqtt
    networks:
      - factory-net

  cloud-subscriber:
    image: eclipse-mosquitto:2
    container_name: lab3-cloud-subscriber
    command: ["sh", "-lc", "echo 'Subscribing to factory/alerts on cloud-mqtt...' && mosquitto_sub -h cloud-mqtt -t factory/alerts -v"]
    depends_on:
      - cloud-mqtt
    networks:
      - factory-net

networks:
  factory-net:
    name: factory-net
```


Проверка

```
PS D:\Documents\Cloud-and-fog-computing\lab3> docker compose up --build
```

Список запустившихся контейнеров

```
[+] Running 6/6
✔ Network factory-net      Created      0.1s
✔ Container lab3-cloud-mqtt Created      1.6s
✔ Container lab3-edge-mqtt Created      1.5s
✔ Container lab3-edge-sensor Created      0.4s
✔ Container lab3-nodered   Created      0.4s
✔ Container lab3-cloud-subscriber Created      0.4s
```

В логах видим что облако получает не все запросы, а лишь те, в которых `temperature > 80`

```
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 59.8, "ts": "2025-12-18T19:15:31.546202+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 68.6, "ts": "2025-12-18T19:15:32.546513+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 74.85, "ts": "2025-12-18T19:15:33.546948+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 75.22, "ts": "2025-12-18T19:15:34.547247+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 76.44, "ts": "2025-12-18T19:15:35.547539+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 93.01, "ts": "2025-12-18T19:15:36.547866+00:00"}
lab3-cloud-subscriber | factory/alerts {"device_id":"sensor-1","temperature":93.01,"ts":"2025-12-18T19:15:36.547866+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 61.6, "ts": "2025-12-18T19:15:37.548148+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 78.74, "ts": "2025-12-18T19:15:38.548444+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 83.61, "ts": "2025-12-18T19:15:39.548745+00:00"}
lab3-cloud-subscriber | factory/alerts {"device_id":"sensor-1","temperature":83.61,"ts":"2025-12-18T19:15:39.548745+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 64.19, "ts": "2025-12-18T19:15:40.549018+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 60.89, "ts": "2025-12-18T19:15:41.549325+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 67.96, "ts": "2025-12-18T19:15:42.549598+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 92.69, "ts": "2025-12-18T19:15:43.549884+00:00"}
lab3-cloud-subscriber | factory/alerts {"device_id":"sensor-1","temperature":92.69,"ts":"2025-12-18T19:15:43.549884+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 64.18, "ts": "2025-12-18T19:15:44.550146+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 67.01, "ts": "2025-12-18T19:15:45.550388+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 83.02, "ts": "2025-12-18T19:15:46.550649+00:00"}
lab3-cloud-subscriber | factory/alerts {"device_id":"sensor-1","temperature":83.02,"ts":"2025-12-18T19:15:46.550649+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 93.86, "ts": "2025-12-18T19:15:47.550893+00:00"}
lab3-cloud-subscriber | factory/alerts {"device_id":"sensor-1","temperature":93.86,"ts":"2025-12-18T19:15:47.550893+00:00"}
lab3-edge-sensor | [edge-sensor] ok {"device_id": "sensor-1", "temperature": 59.37, "ts": "2025-12-18T19:15:48.551196+00:00"}
```

Вывод

В ходе лабораторной работы была смоделирована архитектура Edge/Fog/Cloud для «умной фабрики».

Был реализован Fog-шлюз на базе Node-RED, который выполняет локальную обработку и фильтрацию данных от датчиков.

Экспериментально подтверждено, что использование туманных вычислений значительно сокращает объём данных, передаваемых в облако, и повышает эффективность системы.